goxpyriment — API Reference

Contents

goxpyriment API Reference

This guide documents the complete public API of the `goxpyriment` framework, organized by package.

Note that you can access the source code documentation by running `pkgsite` and opening http://localhost:8080. To install pkgsite:

```
go install golang.org/x/pkgsite/cmd/pkgsite@latest
```

See Viewing the documentation locally for details (and for serving this site offline with zensical).

Package Overview

```
control/      ← experiment lifecycle and orchestration (start here)
stimuli/      ← visual and audio stimulus objects
media/        ← multi-movie playback with hardware-verified display onsets
apparatus/    ← SDL window/renderer, keyboard, mouse, gamepad, gamma
 ↪  corrector
results/      ← experiment data file and output file
design/       ← trial/block structure and randomization
clock/        ← timing utilities
```

```
geometry/     ← coordinate conversion helpers
triggers/     ← hardware trigger devices (EEG sync, etc.)
```

Package `control`

Import: `github.com/chrplr/goxpyriment/control`

Boilerplate

Every experiment starts the same way:

```go
exp := control.NewExperimentFromFlags("My Experiment", control.Black,
 ↪  control.White, 32)
defer exp.End()

err := exp.Run(func() error {
    // trial loop
    return control.EndLoop
})
if err != nil && !control.IsEndLoop(err) {
    log.Fatalf("experiment error: %v", err)
}
```

Pre-experiment Setup Dialog

`GetParticipantInfo` opens a graphical SDL window **before** the experiment starts to collect participant demographics, monitor properties, and display preferences. Call it before `NewExperiment` / `NewExperimentFromFlags`.

```go
fields := append(control.StandardFields, control.FullscreenField)
info, err := control.GetParticipantInfo("My Experiment", fields)
if err != nil {
    log.Fatalf("setup cancelled: %v", err) // user pressed Escape or closed
 ↪  window
}
```

```go
// FieldType selects how a field is rendered.
type FieldType int
const (
    FieldText     FieldType = iota // text input box (default)
    FieldCheckbox                  // tick-box; value is "true" or "false"
)

// InfoField describes one entry in the dialog.
type InfoField struct {
```

```
    Name    string    // key in the returned map
    Label   string    // displayed label
    Default string    // initial value
    Type    FieldType // FieldText (default) or FieldCheckbox
}
```

Types

Pre-built field sets

Variable	Fields
`control.ParticipantFields`	`subject_id`, `age`, `gender`, `handedness`
`control.MonitorFields`	`screen_width_cm`, `viewing_distance_cm`, `refresh_rate_hz`
`control.StandardFields`	`ParticipantFields` + `MonitorFields`
`control.FullscreenField`	Checkbox: `fullscreen` (`"true"` / `"false"`)

Function

Function	Description
`GetParticipantInfo(title string, fields []InfoField) (map[string]string, error)`	Shows the dialog and returns collected values. Returns `ErrCancelled` if the user closes or presses Escape without confirming.

Session persistence All values except `subject_id` are saved to `~/.cache/goxpyriment/last_session.json` on OK and pre-filled on the next run. `subject_id` is always reset.

```
info, err := control.GetParticipantInfo("My Experiment", fields)
// ...
fullscreen := info["fullscreen"] == "true"
width, height := 0, 0
if !fullscreen {
    width, height = 1024, 768
}
exp := control.NewExperiment("My Experiment", width, height, fullscreen,
    control.Black, control.White, 32)

// Set Info (and SubjectID) BEFORE Initialize — they are written to the info
 ↪  file automatically
exp.SubjectID, _ = strconv.Atoi(info["subject_id"])
exp.Info = info
```

```go
if err := exp.Initialize(); err != nil { log.Fatal(err) }
defer exp.End()
```

Using the fullscreen checkbox and persisting to the data file `Initialize()` writes a `--PARTICIPANT INFO` block to the companion `-info.txt` file whenever `exp.Info` is non-nil at that point. No explicit call to `WriteParticipantInfo` is needed.

```go
control.ErrCancelled  // returned when the user cancels the dialog
```

Sentinel error

Constructor Functions

Function	Description
`NewExperimentFromFlags(name string, bg, fg Color, fontSize float32, extra ...InfoField) *Experiment`	Creates and fully initializes an experiment from `-w` (windowed 1024×768), `-d N` (display index, -1 = primary), and `-s N` (subject ID) command-line flags. Calls `log.Fatal` on error. **This is the preferred entry point.** Any `extra` fields are appended to the session-setup dialog that opens when `-s` is absent, and read back from `exp.Info` (nil when the dialog does not open, so keep a flag as the fallback); an extra field named like one of the four built-ins replaces it.
`NewExperiment(name string, width, height int, fullscreen bool, bg, fg Color, fontSize float32) *Experiment`	Lower-level constructor; call `Initialize()` before use.

Lifecycle Methods

Method	Description
`exp.Initialize() error`	Initializes SDL, audio, window, renderer, font, and data file.
`exp.End()`	Cleans up all resources. Always `defer exp.End()` immediately after construction.
`exp.Run(logic func() error) error`	Runs the main trial loop on the SDL main thread. Return `control.EndLoop` to exit cleanly.

Method	Description
`exp.HideCursor() error`	Hides the mouse cursor. `Initialize()` already does this — only needed to hide it again after a `ShowCursor` call.
`exp.ShowCursor() error`	Makes the mouse cursor visible. Call it after `Initialize()` in mouse-driven paradigms; the cursor is hidden by default so it never sits over the stimuli. Equivalent to setting `exp.CursorVisible = true` before `Initialize()`. `GetParticipantInfo` shows the cursor for its own dialog regardless.

Presentation Methods

Method	Description
`exp.Show(stim VisualStimulus) error`	Clear → draw → flip. The standard one-call stimulus presentation.
`exp.ShowTS(stim VisualStimulus) (uint64, error)`	Clear → draw → flip, and return the SDL nanosecond timestamp captured immediately after the VSYNC flip. Use with `GetKeyEventTS` for hardware-precision RT measurement.
`exp.ShowTimed(stim VisualStimulus, durationMs int) error`	`Show(stim)` + `Wait(durationMs)` in one call. For fixation crosses, cues, and passive stimulus viewing.
`exp.ShowFrames(stim VisualStimulus, n int) (uint64, error)`	Hold the stimulus for exactly `n` display frames, returning the onset timestamp of the first flip. The stimulus is redrawn every frame — see *Holding a stimulus for a fixed number of frames* below.
`exp.BlankFrames(n int) (uint64, error)`	Frame-locked counterpart of `Blank(ms)`: clear and hold blank for exactly `n` frames, returning the timestamp of the first flip (i.e. the previous stimulus's offset).
`exp.ShowAndGetRT(stim VisualStimulus, keys []Keycode, timeoutMs int) (Keycode, int64, error)`	Clears stale keyboard events, shows stim with hardware-precise onset timing, waits for a key, and returns `(key, rtMs, error)`. Pass `timeoutMs = -1` for no timeout; returns `(0, 0, nil)` on timeout. This is the canonical single-stimulus RT measurement call.
`exp.ShowEndMessage(message string) error`	Display a centered completion message and wait for any key. For end-of-experiment screens. Laid out by `FittedTextBox`.

Method	Description
`exp.ShowInstructions(text string) error`	Display centered text and wait for spacebar. Laid out by `FittedTextBox`.
`exp.FittedTextBox(text string) *stimuli.TextBox`	A centered `TextBox` wrapped to the drawing area and rendered at the largest point size — never above the default — at which the whole block fits. See *Laying out a block of text* below.
`exp.DrawArea() (w, h float32)`	Size of the coordinate space stimuli are drawn in (the logical resolution). What layout code should measure against.
`exp.Blank(ms int) error`	Clear and flip screen, then wait `ms` milliseconds.
`exp.Wait(ms int) error`	Wait `ms` ms while pumping SDL events (ESC-abortable).
`exp.ShowSplash(waitForKey bool) error`	Show experiment name + version splash.
`exp.Flip() error`	Present the backbuffer and hold to the frame boundary (one call = one display frame).

Input

Method	Description
`exp.Keyboard`	`*apparatus.Keyboard` — see Keyboard section
`exp.Mouse`	`*apparatus.Mouse` — see Mouse section
`exp.PollEvents(handle func(sdl.Event) bool) EventState`	Process all pending SDL events; optionally forward to a handler. Returns `EventState` including nanosecond timestamps.
`exp.HandleEvents() (Keycode, uint32, error)`	Convenience wrapper: returns `(key, mouseButton, error)`.

`EventState` now includes SDL event timestamps:

```
type EventState struct {
    LastKey            sdl.Keycode
    LastMouseButton    uint32
    LastKeyTimestamp   uint64  // SDL nanosecond timestamp of the last key
 ↪  event
    LastMouseTimestamp uint64  // SDL nanosecond timestamp of the last mouse
 ↪  event
    QuitRequested      bool
}
```

Design and Data

Method	Description
`exp.AddDataVariableNames(names []string)`	Register CSV column names for the data file.
`exp.Data.Add(values ...interface{})`	Append a data row. Subject ID is prepended automatically.
`exp.AddBlock(b *design.Block, copies int)`	Add trial blocks to the experiment.
`exp.ShuffleBlocks()`	Randomize block presentation order.
`exp.AddBWSFactor(name string, conditions []interface{})`	Register a between-subjects factor for Latin-square counterbalancing.
`exp.GetPermutedBWSFactorCondition(name string) interface{}`	Return this subject's condition for a BWS factor.
`exp.Design`	`*design.Experiment` — full design object
`exp.Info`	`map[string]string` — values from `GetParticipantInfo`; set before `Initialize()` to persist them automatically to the `-info.txt` file
`exp.CursorVisible`	`bool` — whether the mouse pointer is shown over the experiment window. Defaults to `false`: `Initialize()` hides the cursor. Set it to `true` before `Initialize()` (or call `ShowCursor()` after) for mouse-driven paradigms.

Font and Display

Method	Description
`exp.LoadFont(path string, size float32) error`	Load a TTF font from file.
`exp.LoadFontFromMemory(data []byte, size float32) error`	Load a TTF font from a byte slice.
`exp.SetVSync(vsync int) error`	Toggle vertical sync (1 = on, 0 = off).
`exp.SetLogicalSize(w, h int32) error`	Set device-independent logical resolution.
`exp.SetOutputDirectory(dir string)`	Override default data file directory (`~/goxpy_data`).

Gamma Correction

Standard monitors apply a power-law transfer function L(V) = k·(V/255)^γ ($\gamma \approx 2.2$ for sRGB displays). Equal steps in RGB values do **not** produce equal steps in physical luminance. Use `SetGamma` to enable inverse-gamma correction.

Method	Description
`exp.SetGamma(gamma float64)`	Install a uniform inverse-gamma corrector. Call once after `Initialize()`.
`exp.CorrectColor(c sdl.Color) sdl.Color`	Apply gamma correction to a color. Returns c unchanged when no corrector is set.
`exp.GammaCorrector`	*`apparatus.GammaCorrector` — set directly for per-channel calibration.

```
// Uniform gamma (typical sRGB monitor)
exp.SetGamma(2.2)

// Per-channel gamma (from photometer measurements)
exp.GammaCorrector = apparatus.NewGammaCorrector(2.1, 2.2, 2.3)

// Use in trial loop — specify colors in linear luminance space (0–255)
disk := stimuli.NewFilledCircle(exp.CorrectColor(control.RGB(128, 128,
 ↪  128)), radius)
```

The `apparatus.GammaCorrector` type is also available directly:

```
gc := apparatus.NewGammaCorrectorUniform(2.2)
corrected := gc.CorrectColor(sdl.Color{R: 128, G: 128, B: 128, A: 255})
// corrected.R ≈ 186 — the physical digital value for 50% luminance on γ=2.2
```

Colors, Types, and Constants

```
// Named colors
control.Black, White, Red, Green, Blue, Yellow, Magenta, Cyan
control.Gray, DarkGray, LightGray

// Type aliases (so you only need to import "control")
type Color   = sdl.Color
type Keycode = sdl.Keycode
type FPoint  = sdl.FPoint
type FRect   = sdl.FRect

// Constructors
control.RGB(r, g, b uint8) Color
control.RGBA(r, g, b, a uint8) Color
control.Point(x, y float32) FPoint
control.Origin() FPoint  // returns (0, 0)

// Font helpers
control.FontFromFile(path string, size float32) (*ttf.Font, error)
control.FontFromMemory(data []byte, size float32) (*ttf.Font, error)
```

```
// Loop control
control.EndLoop          // sentinel error: return from Run callback to exit
control.IsEndLoop(err)   // test whether an error is the EndLoop sentinel

// Keyboard codes
control.K_SPACE, K_ESCAPE, K_RETURN, K_BACKSPACE, K_TAB
control.K_UP, K_DOWN, K_LEFT, K_RIGHT
control.K_A … K_Z                                    // full alphabet
control.K_0 … K_9                                    // digit row
control.K_KP_0 … K_KP_9, K_KP_ENTER, K_KP_PLUS, K_KP_MINUS   // numeric
 ↪  keypad
control.K_MINUS, K_PLUS, K_EQUALS, K_LEFTBRACKET, K_RIGHTBRACKET  //
 ↪  punctuation

// Mouse buttons
control.BUTTON_LEFT, BUTTON_RIGHT
```

> **Tip:** The full alphabet, digit row, keypad, and common punctuation keys are re-exported, so experiment code never needs to import `go-sdl3` just to name a key. For a rarely-used code not listed in `defaults.go`, import `go-sdl3/sdl` directly and use `sdl.K_…`.

Audio

```
exp.AudioDevice  // sdl.AudioDeviceID — pass to Sound.PreloadDevice()

// Top-level helper (call before NewExperiment)
control.SetAudioSampleFrames(frames int)  // set audio buffer size
 ↪  (256–2048)
```

Microphone

```
exp.Microphone  // *apparatus.Microphone — nil until OpenMicrophone is
 ↪  called

// Open the default recording device (nil = F32LE mono 44100 Hz)
exp.OpenMicrophone(spec *sdl.AudioSpec) error

// Closed automatically by exp.End()
```

Package `stimuli`

Import: `github.com/chrplr/goxpyriment/stimuli`

Interfaces

```
type Stimulus interface {
    Present(screen *apparatus.Screen, clear, update bool) error
    Preload() error
    Unload() error
}

type VisualStimulus interface {
    Stimulus
    Draw(screen *apparatus.Screen) error
    GetPosition() sdl.FPoint
    SetPosition(pos sdl.FPoint)
}
```

GPU textures are **lazily allocated** on the first `Draw`. To force early allocation (for timing-sensitive code), use:

```
stimuli.PreloadVisualOnScreen(screen, stim)     // single stimulus
stimuli.PreloadAllVisual(screen, []VisualStimulus{...})  // slice
```

Visual Stimuli

Text

Constructor	Description
`NewTextLine(text string, x, y float32, color Color) *TextLine`	Single line of text.
`NewTextBox(text string, width int32, pos FPoint, color Color) *TextBox`	Word-wrapped multi-line text.

Both support a `Font *ttf.Font` field — set it to override the screen default.

Shapes

Constructor	Description
`NewFixCross(size, lineWidth float32, color Color) *FixCross`	Fixation cross centered at (0, 0).
`NewCircle(radius float32, color Color) *Circle`	Filled circle.
`NewRectangle(cx, cy, w, h float32, color Color) *Rectangle`	Filled rectangle.
`NewLine(x1, y1, x2, y2 float32, color Color) *Line`	Line segment.

Images and Video

Constructor/Function	Description
`NewPicture(filePath string, x, y float32) *Picture`	Image loaded from file (PNG, JPG, BMP…).
`NewPictureFromMemory(data []byte, x, y float32) *Picture`	Image loaded from embedded bytes.
`NewSpriteSheet(filePath string) *SpriteSheet`	One image holding many stimuli, sharing a single GPU texture.
`NewSpriteSheetFromMemory(data []byte) *SpriteSheet`	Same, from embedded bytes (required for the browser build).
`(*SpriteSheet) Grid(screen, cols, rows int) ([]*Sprite, error)`	Cut into `cols*rows` equal cells, row-major.
`(*SpriteSheet) GridWithSpacing(screen, cols, rows int, margin, spacing float32) ([]*Sprite, error)`	As `Grid`, for sheets with a border or gutters.
`(*SpriteSheet) Sprites(screen, clips []sdl.FRect) ([]*Sprite, error)`	Explicit source rectangles, for irregular sheets.
`(*SpriteSheet) Unload() error`	Destroys the shared texture. `Sprite.Unload` is a no-op by design.
`PlayGv(screen, path string, x, y float32) ([]UserEvent, error)`	Play a `.gv` (LZ4-compressed RGBA) video file, VSYNC-locked.
`NewGvVideo(path string) (*GvVideo, error)`	Open a `.gv` file for frame-by-frame access.

Psychophysics Stimuli

Constructor	Description
`NewGaborPatch(sigma, theta, lambda, phase, psi, gamma float64, bgColor Color, size float32) *GaborPatch`	Static Gabor patch. `theta` in degrees, `lambda` = spatial wavelength in pixels.
`NewDotCloud(radius float32, bgColor, dotColor Color) *DotCloud`	Static random-dot cloud. Call `Make(nDots, dotRadius, gap)` to populate.
`NewRDS(imgSize, innerSize [2]int, shift, gap, scale int) *RDS`	Random-dot stereogram (side-by-side pair).
`NewVisualMask(w, h, dotW, dotH float32, bgColor, dotColor Color, pct int) *VisualMask`	Random-dot masking stimulus. `pct` = dot fill percentage 0–100.

Composite / Interactive

Constructor	Description
`NewThermometerDisplay(size FPoint, nSegments int, state, goal float32) *ThermometerDisplay`	Segmented progress bar. `State` and `Goal` in 0–100.
`NewChoiceGrid(choices []string, maxSelect int, prompt string) *ChoiceGrid`	Multiple-choice button grid (mouse + keyboard). See below.
`NewTextInput(msg string, pos FPoint, boxW float32, bgColor, frameColor, textColor Color) *TextInput`	Free-text keyboard input box. Call `ti.Get(screen, keyboard)`.
`NewMenu(items []string) *Menu`	Numbered keyboard-navigable list. Call `m.Get(screen, keyboard, initialSel)`.

ChoiceGrid

```
cg := stimuli.NewChoiceGrid(choices, maxSelect, prompt)
cg.Cols = 7       // optional: set column count (0 = auto)

selections, err := cg.Get(exp.Screen, exp.Keyboard)
// selections is a []string preserving selection order
```

- `MaxSelect > 0`: auto-submits after N selections.
- `MaxSelect == 0`: participant presses ENTER or SPACE to submit.
- BACKSPACE removes the last selection.
- Both mouse click and matching keypress (single-char labels) activate buttons.

Menu

```
m := stimuli.NewMenu([]string{"Option A", "Option B", "Option C"})
m.Pos = sdl.FPoint{X: 0, Y: 0}    // optional: reposition (default = screen
 ↪  center)
m.HighlightColor = control.Yellow  // optional: override highlight color

idx, err := m.Get(exp.Screen, exp.Keyboard, 0)  // 0 = initially highlight
 ↪  first item
// idx is 0-based; -1 + sdl.EndLoop on ESC/quit
```

Navigation: UP/DOWN arrows move the highlight; ENTER or SPACE confirms; number keys 1–9 (0 for tenth) select and confirm directly. The selected item is shown in `HighlightColor` with a > prefix; others use `TextColor`. `LineSpacing` controls vertical item spacing (0 = auto from font height).

Animated / VSYNC-locked Loops

All three functions disable GC, lock to VSYNC, and return `(MotionResult, error)`.

```
type MotionResult struct {
    Key    sdl.Keycode // interrupt key pressed (0 if none)
    Button uint8       // mouse button pressed (0 if none)
    RTms   int64       // ms from first frame to response (or total duration
 ↪  on timeout)
}
```

Function	Description
`PresentMovingDotCloud(screen, nDots int, dotRadius, cloudRadius float32, center FPoint, speedPxPerSec float32, maxDurationMs int64, interruptKeys []Keycode, catchMouse bool, dotColor, bgColor Color) (MotionResult, error)`	Animated random-dot cloud. Each dot moves at a fixed speed and respawns when it exits the boundary.
`PresentMovingGrating(screen, width, height float32, center FPoint, orientation, spatialFreq, temporalFreq, contrast, bgLuminance float64, maxDurationMs int64, interruptKeys []Keycode, catchMouse bool) (MotionResult, error)`	Drifting sinusoidal grating in a rectangular aperture.
`PresentMovingGabor(screen, size float32, sigma float64, center FPoint, orientation, spatialFreq, temporalFreq, contrast, bgLuminance float64, maxDurationMs int64, interruptKeys []Keycode, catchMouse bool) (MotionResult, error)`	Drifting Gabor patch with Gaussian envelope (alpha-blended edges).

Spatial frequency is in **cycles per pixel** (e.g. `0.05` = one cycle every 20 px). Temporal frequency is in **Hz**. Orientation is in **degrees from horizontal** (0° = vertical bars drifting right).

Stimulus Streams (High-Precision RSVP)

Stream functions disable GC, lock every onset and offset to a VSYNC boundary, and return `([]UserEvent, []TimingLog, error)`.

```
type UserEvent struct {
    Event       sdl.Event     // raw SDL event (KeyboardEvent,
 ↪  MouseButtonEvent, …)
    Timestamp   time.Duration // time relative to stream start (Go clock, ms
 ↪  precision)
    TimestampNS uint64        // SDL3 hardware event timestamp, nanoseconds
 ↪  (same clock as Screen.FlipTS)
}
```

```
type TimingLog struct {
    Index        int
    TargetOn     time.Duration
    ActualOnset  time.Duration // DIAGNOSTIC (Go clock): first-frame draw,
 ↪  stream-relative; not for RT
    ActualOffset time.Duration // DIAGNOSTIC (Go clock): after last
 ↪  on-frame, stream-relative; not for RT
    OnsetNS      uint64        // AUTHORITATIVE: SDL3 ns timestamp
 ↪  (sdl.TicksNS clock) of the stimulus onset
    OffsetNS     uint64        // AUTHORITATIVE: SDL3 ns timestamp
 ↪  (sdl.TicksNS clock) of the stimulus offset
}
```

The `OnsetNS`/`OffsetNS` pair is the authoritative timing record: both are on the SDL3 nanosecond clock (`sdl.TicksNS()`) — the same clock that stamps input events (`UserEvent.TimestampNS`, `Keyboard.GetKeyEventTS`) and VSYNC flips (`Screen.FlipTS`). A reaction time or a displayed duration is a plain subtraction within this one clock: `int64(event.TimestampNS - l.OnsetNS)`, or `int64(l.OffsetNS - l.OnsetNS)`.

`ActualOnset`/`ActualOffset` are Go-clock (`time.Now`) **diagnostics only**, kept for coarse pacing checks. They live on a different timebase (different origin) from the NS fields and from input events, so they must never be subtracted from an event timestamp or logged as a canonical onset/offset. Use the NS fields for any real timing.

`OnsetNS` is zero only when a stimulus was never displayed (zero on-frames); `OffsetNS` is the VSYNC timestamp of the flip that takes the stimulus down, and for the final element of a contiguous stream (which has no ISI flip of its own) it is synthesised from the onset plus the on-frame count so that it, too, is on the SDL clock rather than one frame early.

`UserEvent.TimestampNS` and `TimingLog.OnsetNS`/`OffsetNS` are all on the SDL3 nanosecond clock, so reaction times measured during a stream can be computed with full hardware precision:

```
for _, ev := range events {
    if ev.Event.Type == sdl.EVENT_KEY_DOWN {
        // Find the stimulus that was on-screen when the key was pressed
        for _, l := range logs {
            if ev.TimestampNS >= l.OnsetNS && ev.TimestampNS < l.OffsetNS {
                rtNS := int64(ev.TimestampNS - l.OnsetNS)
                fmt.Printf("RT from stimulus %d: %d ms\n", l.Index, rtNS/1_
 ↪  000_000)
            }
        }
    }
}
```

Searching event lists

Function	Description
`FirstKeyPress(events []UserEvent, key sdl.Keycode) (UserEvent, bool)`	Returns the first `KEY_DOWN` event matching key from the slice, plus a found flag.

```
if ev, ok := stimuli.FirstKeyPress(events, sdl.K_SPACE); ok {
    fmt.Printf("Space pressed at %d ms\n", ev.Timestamp.Milliseconds())
}
```

```
// RSVP text stream — simplest entry point
events, logs, err := stimuli.PresentStreamOfText(
    exp.Screen, words, durationOn, durationOff, x, y, color,
)

// Image/mixed stream
elements := stimuli.MakeRegularVisualStream(stims, durationOn, durationOff)
events, logs, err := stimuli.PresentStreamOfImages(exp.Screen, elements, x,
 ↪  y)

// Irregular timing
elements, err := stimuli.MakeVisualStream(stims, onsetMs, durationMs)
events, logs, err := stimuli.PresentStreamOfImages(exp.Screen, elements, x,
 ↪  y)
```

Visual Streams

```
// Regular timing
elements := stimuli.MakeRegularSoundStream(sounds, durationOn, durationOff)
events, logs, err := stimuli.PlayStreamOfSounds(elements)

// Irregular timing
elements, err := stimuli.MakeSoundStream(sounds, onsetMs, durationMs)
events, logs, err := stimuli.PlayStreamOfSounds(elements)
```

Audio Streams `sounds` is `[]stimuli.AudioPlayable` — satisfied by both `*Sound` and `*Tone`. The returned `TimingLog` carries `OnsetNS`/`OffsetNS` on the SDL3 clock (captured at the `Play()` instant and the end of the on-phase), so audio-stream onsets are directly comparable with input-event timestamps, same as visual streams.

Mixed Streams `PresentStreamOfStimuli` presents a heterogeneous, **sequential** stream that freely mixes visual and audio stimulus types (anything satisfying the broad `Stimulus` interface). It uses the same VSYNC-locked, GC-disabled loop as `PresentStreamOfImages` (which now delegates to it).

```
elements := []stimuli.StreamElement{
    {Stimulus: stimuli.NewTextLine("Ready?", 0, 0, color), DurationOn: 600 *
     ↪  time.Millisecond, DurationOff: 200 * time.Millisecond},
    {Stimulus: pic,  DurationOn: 800 * time.Millisecond}, // held while the
     ↪  tone plays
    {Stimulus: tone, DurationOn: 400 * time.Millisecond, DurationOff: 200 *
     ↪  time.Millisecond},
}
events, logs, err := stimuli.PresentStreamOfStimuli(exp.Screen, elements,
 ↪  x, y)

// Builders mirror the visual/sound ones but take []stimuli.Stimulus:
elements = stimuli.MakeRegularStream(stims, durationOn, durationOff)
elements, err = stimuli.MakeStream(stims, onsetMs, durationMs)
```

Per-element semantics:

- **Visual** elements (those satisfying `VisualStimulus`) are centered on `(x, y)` and redrawn every frame for `DurationOn`, then blanked for `DurationOff`.
- **Audio / non-visual** elements (and a `nil` Stimulus) are triggered once, right after the slot's first VSYNC flip, and **do not clear the screen** — the previous frame is held for the whole slot. Place a visual just before a sound to keep it on screen while the sound plays. `TimingLog.OnsetNS` is the VSYNC reference at which the audio was triggered.

Audio elements must already be bound to the device via `PreloadDevice(exp.AudioDevice)` before the call (same precondition as `PlayStreamOfSounds`). For pure-audio streams prefer `PlayStreamOfSounds`, whose sub-frame sleep-polling gives finer timing than 60 Hz frame quantization.

Per-frame callback `PresentStreamOfStimuliFunc` adds an optional `FrameCallback` invoked once per frame, **just before each flip**. Use it to draw a persistent overlay (trial counter, frame border, fixation) on top of the stimulus, or to run real-time logic such as firing feedback the instant a response window lapses — things the plain stream functions cannot express.

```
header := stimuli.NewTextLine("3 / 40", 0, -300, control.White)
cb := func(ctx stimuli.FrameContext) error {
    _ = header.Draw(ctx.Screen)           // overlay, rendered over the
 ↪  stimulus
    if ctx.OnPhase && ctx.FirstFrame {    // onset of element ctx.Index
        // real-time feedback using ctx.NowNS / ctx.Events;
        // return sdl.EndLoop to stop early, any other error to abort
    }
    return nil
}
events, logs, err := stimuli.PresentStreamOfStimuliFunc(exp.Screen,
 ↪  elements, x, y, cb)
```

`FrameContext` fields: `Screen`, `Index`, `Frame`, `OnPhase`, `FirstFrame`, `NowNS` (pre-flip `sdl.TicksNS()`), `Elapsed`, `Events` (through the previous frame). Passing `nil` for the callback is identical to `PresentStreamOfStimuli`.

Per-stimulus onset trigger (post-flip hook) To emit a hardware TTL — or run any action — aligned to each stimulus **onset**, use `PresentStreamOfStimuliHooks`, which adds an optional `OnsetCallback` to the per-frame callback:

```
func PresentStreamOfStimuliHooks(
    screen *apparatus.Screen, elements []StreamElement, x, y float32,
    onFrame FrameCallback, onOnset OnsetCallback,
) ([]UserEvent, []TimingLog, error)

type OnsetCallback func(index int, onsetNS uint64) error
```

`onOnset` fires once per element with a stimulus onset, **immediately after** the VSYNC flip that turns it on (for a held audio element whose `DurationOn` rounds to zero frames, after the first-off-frame flip that triggers it). `index` matches the returned `TimingLog` slice and `onsetNS` equals `TimingLog[index].OnsetNS`.

This is the **post-flip** counterpart to `FrameCallback`: `FrameCallback` runs one frame earlier (pre-flip, at GPU-submission time), whereas `OnsetCallback` runs on the photon side of the frame boundary, so a trigger emitted here coincides with the logged onset instead of leading it by a frame.

```
dev := triggers.NewParallelPort("/dev/parport0") // any
 ↪  triggers.OutputTTLDevice
if err := dev.Open(); err != nil {
    log.Fatal(err)
}
defer dev.Close()

onset := func(index int, onsetNS uint64) error {
    return dev.Send(codes[index])        // per-stimulus code, at the
     ↪  displayed onset
}
clear := func(ctx stimuli.FrameContext) error {
    if !ctx.OnPhase && ctx.FirstFrame {  // drop the lines at the start of
     ↪  each ISI
        return dev.AllLow()
    }
    return nil
}
events, logs, err := stimuli.PresentStreamOfStimuliHooks(exp.Screen,
 ↪  elements, 0, 0, clear, onset)
```

The hook runs on the timing-critical path with GC disabled, between the onset flip and the next frame, so it **must be short and non-blocking**: emit an edge (`SetHigh` / `Send`) here and clear it from a later `FrameCallback` or after the stream — never `time.Sleep`

or a blocking `Pulse`, or a frame is missed. Elements with a `nil` Stimulus (pure delay / hold slots) have no onset and do not fire it. Either callback may be `nil`; `PresentStreamOfStimuliFunc` is exactly `PresentStreamOfStimuliHooks(..., onFrame, nil)`.

For pure-audio streams the equivalent is `PlayStreamOfSoundsHook(elements, onOnset)`, which fires `onOnset` once per non-nil `Sound`, immediately after `Play()`, with the same SDL-clock `onsetNS` (`== TimingLog[index].OnsetNS`). `PlayStreamOfSounds` is `PlayStreamOfSoundsHook(elements, nil)`.

Audio Stimuli

```
// WAV file
snd := stimuli.NewSound(filePath)
snd.PreloadDevice(exp.AudioDevice)
snd.Play()
snd.Wait()                                      // block until done
snd.PlaySegment(onset, offset, rampSec)        // time-delimited segment with
↪  optional fade

// Embedded WAV
snd := stimuli.NewSoundFromMemory(data)

// Procedural tone
tone := stimuli.NewTone(frequency, duration, volume)   // duration:
↪  time.Duration; volume: 0–255
tone.PreloadDevice(exp.AudioDevice)
tone.Play()

// One-shot helper (no preload needed)
stimuli.PlaySoundFromMemory(exp.AudioDevice, data)

// Embedded feedback sounds (via assets_embed)
import "github.com/chrplr/goxpyriment/assets_embed"
stimuli.PlaySoundFromMemory(exp.AudioDevice, assets_embed.BuzzerWav)
stimuli.PlaySoundFromMemory(exp.AudioDevice, assets_embed.CorrectWav)
```

Package `apparatus` and `results`

Import: `github.com/chrplr/goxpyriment/apparatus` (screen, input, gamma) Import: `github.com/chrplr/goxpyriment/results` (data file)

In normal experiments you access `apparatus` types through `exp.Screen`, `exp.Keyboard`, `exp.Mouse`, and `exp.Data`. Direct use of `apparatus` is only needed when writing custom stimulus types.

Screen

All stimulus positions use a **center-origin coordinate system**: `(0, 0)` is the screen center; positive Y is upward.

`screen.Width` and `screen.Height` are the **logical** drawing space — the coordinate system stimuli are positioned in — and move with `SetLogicalSize`. They are what layout code should measure against; a width computed from anything else lands somewhere other than where `CenterToSDL` puts it. For *physical* pixels, as in a degrees-of-visual-angle calculation, ask `screen.Renderer.CurrentOutputSize()` instead. In fullscreen with no logical size set the two are the same, which is what makes the distinction easy to miss.

```
screen.CenterToSDL(x, y float32) (float32, float32)  // convert to SDL
 ↪  top-left coords
screen.CenteredRect(pos FPoint, w, h float32) *FRect  // SDL dest rect for a
 ↪  w×h texture centered at pos
screen.MousePosition() (float32, float32)             // current cursor in
 ↪  center coords
screen.Clear() error                                  // fill with
 ↪  background color
screen.Update() error                                 // present + hold to
 ↪  the frame boundary
screen.Flip() error                                   // alias for Update
screen.FlipTS() (uint64, error)                       // present + return
 ↪  SDL nanosecond timestamp after flip
screen.FrameDuration() time.Duration                  // nominal frame
 ↪  duration (falls back to 60 Hz)
screen.CalibrateRefresh(n int) (time.Duration, error)  // measured frame
 ↪  period, pacing bypassed
screen.SetLogicalSize(w, h int32) error               // also updates
 ↪  screen.Width/Height
screen.SetVSync(vsync int) error
screen.DisplayInfo() apparatus.DisplayInfo                  // monitor
 ↪  properties
screen.Destroy()
```

Laying out a block of text

`ShowInstructions`, `ShowEndMessage` and `FittedTextBox` size a text screen by measuring it, not by a fixed fraction of the window. Two things go wrong when a wrap width is written as a constant share of the screen:

- **The column count moves with the display.** A wrap width in pixels divided by a font in points is a number of characters, and it changes per machine: at 28 pt in the default monospace font, 80 % of a 1024-pixel-wide screen is 48 characters and 80 % of a 1920-pixel one is 90. Instruction text hand-wrapped in the source at the usual 55–70 columns therefore survives on the author's screen and is re-broken on a narrower one, each over-long line becoming a full line plus a short

orphan.
- **Nothing checks the height.** A screen that has grown a few lines simply runs off the bottom edge, with no error and nothing in the log.

`FittedTextBox` wraps at `0.92 × DrawArea().w` and then picks the largest point size, up to `exp.DefaultFontSize`, at which the rendered block fits within `0.90 × DrawArea().h`. Where it can do so without shrinking below `keepBreaksFloor` (three quarters) of that size, it prefers a size at which no line the author wrote has to be re-wrapped — so hand-wrapped text keeps its own breaks on any display, while a paragraph written as one long line still wraps, as intended. Below `control.MinTextFontSize` (11 pt) it stops and logs a warning: a screen that will not fit at 11 pt is too long, not too big.

The fitted font belongs to the experiment and is closed by `End()`; a caller of `FittedTextBox` only has to `Unload()` the box.

`FlipTS` returns `sdl.TicksNS()` captured immediately after the flip. This timestamp is on the same nanosecond clock as SDL3 event timestamps, so `int64(event.Timestamp - onsetNS)` gives hardware-precision reaction time without any polling latency.

`Update` (and therefore `Flip`, `FlipTS`, `Show`, `ShowTS`) presents **and then holds to the frame boundary**, so one call always occupies exactly one display frame. This is unconditional because `SDL_RenderPresent` cannot be trusted to block until the retrace — under triple/mailbox buffering it returns immediately — and the wait costs nothing where the driver does block. Pacing is skipped when VSync is off.

`CalibrateRefresh` bypasses that pacing and presents directly, so it reports the *unaided* driver behaviour; compare it against `FrameDuration`. `Initialize` runs it over 60 frames at startup and writes `sys refresh_nominal_hz` / `sys refresh_measured_hz` into the session's `-info.txt`, warning if they differ by more than 10%. Measured slower than nominal means frames are being dropped before the panel, which pacing cannot fix.

Holding a stimulus for a fixed number of frames There is no "wait for *n* VSYNC edges" call, and there cannot be a useful one: SDL3 exposes no `SDL_WaitVBlank`, so the only way to stay locked to the display is to present a frame — and a frame carrying no draw calls is not reliably scanned out under a compositor (see *Never present a frame with no draw calls* in `apparatus/CLAUDE.md`). A hold must therefore **redraw its content once per frame**:

```
for i := 0; i < n; i++ {
    screen.Clear()
    stim.Draw(screen)
    screen.Flip()
}
```

`exp.ShowFrames(stim, n)` and `exp.BlankFrames(n)` wrap that loop and return the onset timestamp of the first flip:

```
onsetNS, _ := exp.ShowFrames(rect, 10)   // 10 frames on screen
offsetNS, _ := exp.BlankFrames(10)       // 10 frames blank
exp.Data.Add(onsetNS, offsetNS)
```

There is no “wait for n frames” call that skips the redraw. A hold must redraw its content once per frame: SDL invalidates the backbuffer on every present, so re-clearing with whatever draw colour was last set paints the screen in that colour instead of holding the stimulus. Use the two calls above.

Keyboard

```
key, err := exp.Keyboard.Wait()                                    // any key
key, err := exp.Keyboard.WaitKey(control.K_SPACE)                 // specific
 ↪  key
key, err := exp.Keyboard.WaitKeys(keys, timeoutMS)                 // first
 ↪  of several keys (-1 = no timeout)
key, rt, err := exp.Keyboard.WaitKeysRT(keys, timeoutMS)           // with RT
 ↪  in ms from call site
key, ts, err := exp.Keyboard.GetKeyEventTS(keys, timeoutMS)        // with
 ↪  SDL event timestamp (nanoseconds)
events, err := exp.Keyboard.GetKeyEventsTS(keys, timeoutMS)        // first
 ↪  key + 50 ms window; for bilateral responses
events, err := exp.Keyboard.CollectKeyEventsTS(keys, durationMS)  // all
 ↪  keys during full fixed window
key, err := exp.Keyboard.Check()                                   //
 ↪  non-blocking poll
held := exp.Keyboard.IsPressed(key)                               // true if
 ↪  key is physically held now
upTS, err := exp.Keyboard.WaitKeyReleaseTS(key, timeoutMS)         // wait
 ↪  for KEY_UP; returns hardware timestamp
exp.Keyboard.Clear()                                              // drain SDL
 ↪  event queue
```

`WaitKeys` and `WaitKeysRT` return `0, nil` on timeout; return `sdl.EndLoop` on ESC or window close.

`IsPressed` queries SDL’s scancode state array — no event queue involvement. `WaitKeyReleaseTS` returns the KEY_UP hardware timestamp so that `upTS - downTS` gives nanosecond-precision press duration.

`GetKeyEventsTS` — two-phase collection for bilateral responses. The timeout governs how long to wait for the *first* key. After the first key arrives, the function waits an additional 50 ms for any second key before returning. This extra window is necessary because human “simultaneous” bilateral presses (e.g. both hands at once) arrive 10–50 ms apart — a non-blocking drain after the first key would miss the second key almost every time. Use `GetKeyEventTS` for ordinary single-key trials to avoid the 50 ms overhead.

`GetKeyEventTS` returns the SDL3 `KeyboardEvent.Timestamp` field — the nanosecond time at which the hardware key-down event was generated, on the same clock as `sdl.TicksNS()` and `Screen.FlipTS()`. This allows computing reaction time from any specific stimulus onset without manual arithmetic:

```
onset, _ := exp.ShowTS(stim1)    // nanoseconds at VSYNC flip
exp.Wait(500)
exp.ShowTS(stim2)
key, eventTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, -1)
rtToStim1 := int64(eventTS - onset)  // nanoseconds
```

Mouse

```
x, y := exp.Mouse.Position()                                  // current position
 ↪  (center coords)
btn, err := exp.Mouse.WaitPress()                              // block until
 ↪  button pressed
btn, rt, err := exp.Mouse.WaitPressRT(timeoutMS)               // with RT in ms
 ↪  from call site
btn, ts, err := exp.Mouse.GetPressEventTS(timeoutMS)           // with SDL event
 ↪  timestamp (nanoseconds)
btn, err := exp.Mouse.Check()                                 // non-blocking poll
held := exp.Mouse.IsPressed(sdl.BUTTON_LEFT)                   // true if button
 ↪  is physically held now
upTS, err := exp.Mouse.WaitButtonReleaseTS(btn, timeoutMS) // wait for
 ↪  MOUSE_BUTTON_UP; hardware timestamp
exp.Mouse.ShowCursor(show bool) error
```

`WaitPressRT` mirrors `Keyboard.WaitKeysRT`: reaction time is measured in milliseconds from the call site. `GetPressEventTS` returns the SDL3 hardware event timestamp in nanoseconds, suitable for use with `ShowTS`. `IsPressed` and `WaitButtonReleaseTS` mirror the keyboard's `IsPressed` and `WaitKeyReleaseTS`.

GamePad

```
pads, err := apparatus.GetGamePads()                                    //
 ↪  enumerate connected gamepads
defer pads[0].Close()

btn, err := pads[0].WaitPress()                                  // block until
 ↪  any button
btn, ts, err := pads[0].GetPressEventTS(timeoutMS)              // with SDL
 ↪  event timestamp (nanoseconds)
```

`GetPressEventTS` returns the `GamepadButtonEvent.Timestamp` field — same nanosecond clock as `Screen.FlipTS` and keyboard/mouse event timestamps.

Unified Input — `WaitAnyEventTS`

When the response device is not fixed in advance (keyboard *or* mouse click), use the method on `Experiment`:

```
// Accept F or J key, or any mouse button, timeout after 3 s
ev, err := exp.WaitAnyEventTS(
    []control.Keycode{control.K_F, control.K_J},
    true,   // catchMouse
    3000,
)
```

Returns an `apparatus.InputEvent`:

```
type InputEvent struct {
    Device       apparatus.DeviceKind     // DeviceKeyboard | DeviceMouse |
 ↪  DeviceGamepad
    Key          sdl.Keycode       // non-zero for keyboard events
    Button       uint32             // non-zero for mouse events
    GamepadButton sdl.GamepadButton // non-zero for gamepad events
    TimestampNS   uint64            // SDL3 hardware timestamp, nanoseconds
}
```

`TimestampNS` is on the same clock as `ShowTS`, so RT computation is identical regardless of device:

```
onset, _ := exp.ShowTS(stim)
ev, _ := exp.WaitAnyEventTS(keys, true, -1)
rtNS := int64(ev.TimestampNS - onset)
```

Pass `keys = nil` to accept any key. Pass `catchMouse = false` to ignore the mouse. On timeout, returns a zero `InputEvent` and `nil` error. On ESC or quit, returns `sdl.EndLoop`.

ResponseDevice

`ResponseDevice` is a unified interface over all participant-input hardware — SDL-event-driven devices (keyboard, mouse, gamepad) **and** polled TTL devices (MEGTTLBox, DLPIO8). It is the recommended abstraction when the experiment design does not commit to a specific input modality.

```
type ResponseDevice interface {
    WaitResponse(ctx context.Context) (Response, error)
    DrainResponses(ctx context.Context) error
    Close() error
}

type Response struct {
    Source  apparatus.DeviceKind  // DeviceKeyboard | DeviceMouse |
 ↪  DeviceGamepad | DeviceTTL
    Code    uint32         // SDL Keycode, mouse button, gamepad button, or
 ↪  TTL bitmask
    RT      time.Duration  // elapsed from WaitResponse call to detection
    Precise bool           // true = hardware event timestamp; false =
 ↪  software poll
```

```
}
```

`Response.Precise` distinguishes two timing regimes:

Device	Precise	RT origin
Keyboard, Mouse, Gamepad	`true`	SDL3 hardware event timestamp (nanosecond)
MEGTTLBox, DLPIO8	`false`	`time.Now()` at poll detection (±poll interval, ~5 ms)

Construct wrappers with the provided adapters:

```
// SDL-event-driven devices
rd := &apparatus.KeyboardResponseDevice{KB: exp.Keyboard}
rd := &apparatus.MouseResponseDevice{M: exp.Mouse}
rd := &apparatus.GamepadResponseDevice{GP: pad}

// Polled TTL device (MEGTTLBox, DLPIO8, or any type with
 ↪  ReadAll/DrainInputs)
box, _ := triggers.NewMEGTTLBox("/dev/ttyACM0")
rd := apparatus.NewTTLResponseDevice(box, 5*time.Millisecond)
```

Usage in a trial loop:

```
onset, _ := exp.ShowTS(stim)
_ = rd.DrainResponses(ctx)
resp, err := rd.WaitResponse(ctx)
// resp.RT is always valid; resp.Precise tells you whether to trust
 ↪  nanosecond accuracy
```

Microphone

```
// Construction
mic, err := apparatus.NewMicrophone(spec)                          // default
 ↪  recording device
mic, err := apparatus.NewMicrophoneFromDevice(devID, spec)          //
 ↪  specific device
apparatus.DefaultRecordingSpec() sdl.AudioSpec                      // F32LE
 ↪  mono 44100 Hz
devices, err := apparatus.GetRecordingDevices() []sdl.AudioDeviceID

// Fields
mic.DeviceID  sdl.AudioDeviceID
mic.Stream    *sdl.AudioStream
mic.Spec      sdl.AudioSpec
```

```
// Methods
captureStartNS, err := mic.StartCapture()    // flush buffer, resume device,
 ↪  return TicksNS()
err := mic.StopCapture()                     // pause device
n, err := mic.ReadSamples(buf []byte)       // read PCM bytes (non-blocking)
n, err := mic.Available()                    // bytes waiting to be read
mic.Close()                                  // destroy stream
```

In normal experiments, open the microphone via `exp.OpenMicrophone(spec)` (see control package); `exp.Microphone` is then available throughout the experiment and closed by `exp.End()`.

VoiceKey

Detects voice onset by computing per-window RMS over F32LE PCM samples.

```
vk := apparatus.NewVoiceKey(mic, threshold float32, windowSz int) *VoiceKey
// threshold: RMS amplitude (0–1). Recommended starting value: 0.02–0.05.
// windowSz:  samples per RMS window. 0 → 128 (≈ 2.9 ms at 44100 Hz).

captureStartNS, err := vk.Arm()   // flush mic buffer, start capture, return
 ↪  TicksNS()

onsetNS, pcm, err := vk.WaitOnset(captureStartNS uint64, timeoutMS int)
 ↪  (uint64, []byte, error)
// Returns: onset timestamp (sdl.TicksNS() domain), all PCM bytes captured,
 ↪  nil or ErrVoiceTimeout.
```

`onsetNS` and the `imageOnsetNS` returned by `exp.ShowTS` or `screen.FlipTS` are on the same SDL3 nanosecond clock:

```
captureStartNS, _ := vk.Arm()
imageOnsetNS, _   := exp.ShowTS(picture)
onsetNS, pcm, _   := vk.WaitOnset(captureStartNS, 2000)
rtMs := int64(onsetNS - imageOnsetNS) / 1_000_000
```

Sentinel error: `apparatus.ErrVoiceTimeout`

Post-hoc helpers (for re-analysing saved WAV files):

```
sampleIdx := apparatus.ScanOnset(pcm []byte, threshold float32, windowSz
 ↪  int) int
// Returns the sample index of the first onset window, or -1 if not found.

onsetNS := apparatus.SampleOnsetNS(captureStartNS uint64, sampleIndex,
 ↪  sampleRate int) uint64
// Converts a sample index back to a nanosecond timestamp.

names, err := apparatus.DeviceNames() map[sdl.AudioDeviceID]string
// Returns all recording devices and their human-readable names.
```

WriteWAV

```
apparatus.WriteWAV(path string, spec sdl.AudioSpec, data []byte) error
```

Saves raw PCM bytes as a standard RIFF/WAV file. Supports `AUDIO_F32LE`, `AUDIO_S16LE`, `AUDIO_S32LE`, and `AUDIO_U8`. Writes atomically via a temp file + rename.

DataFile

```
exp.Data.Add(field1, field2, ...)              // append a data row
exp.Data.AddVariableNames([]string{...})       // write column header
exp.Data.WriteDisplayInfo(info)                // append display metadata to
 ↪  the info file
exp.Data.WriteHostInfo(sysinfo.Host())         // append machine/OS metadata
 ↪  to the info file
exp.Data.WriteParticipantInfo(info)            // append --PARTICIPANT INFO
 ↪  block to the info file (called automatically by Initialize when exp.Info
 ↪  is set)
exp.Data.WriteEndTime()                        // append end time + duration
 ↪  to the info file
```

Two files are written to `~/goxpy_data/` for each session:

File	Contents
`<expname>_sub-<NNN>_date-<YYYYMMDD>-<HHMMSS>.csv`	Pure CSV data rows — directly importable by Excel, R, or pandas
`<expname>_sub-<NNN>_date-<YYYYMMDD>-<HHMMSS>-info.txt`	#-prefixed metadata: start/end time, hostname, OS, framework version, display and audio configuration, participant info

---

Package `media`

Import: `github.com/chrplr/goxpyriment/media`

Multi-movie playback with shared master-clock synchronisation, look-ahead frame conditions, and post-vsync display events for hardware- trigger alignment. Adds `MovieManager` (per-experiment owner of the clock and movie set), `Movie` (per-movie state with PsyScope-Tahoe semantics), and the `media/present` subpackage of OS-level vsync backends. Pure Go, `.gv` movies only, silent.

Full reference (precision contracts, platform support, mapping to PsyScope movie commands): see MediaMovies.md.

Quick API summary

```
mgr := media.NewMovieManager(exp.Screen)
defer mgr.Close()

gv, _ := gvvideo.LoadGVVideo("clip.gv")
mov, _ := media.NewMovie(mgr, gv,
    media.WithTag("M1"),
    media.WithRepeat(-1),
    media.WithSize(640, 360),
    media.WithPosition(sdl.FPoint{X: -200, Y: 0}),
)
defer mov.Close()

// Look-ahead — fires from inside DrawWithoutFlip, BEFORE the frame is
 ↪  presented.
mov.OnAt(media.Frame(186), func(o media.Onset) { /* ... */ })

// Hardware-verified — fires from NotifyFlipped with the OS vsync timestamp.
mov.OnAtDisplay(media.Frame(186), func(o media.Onset) {
    _ = ttl.Pulse(0, 5*time.Millisecond)
})

// Display[Onset/Offset] — fires when the named tag changes visibility.
mgr.OnDisplayOnset("M1", func(o media.Onset) { /* ... */ })

// Atomic command burst (PsyScope script-line equivalent):
mgr.BeginBurst()
movieA.Pause(); movieB.Pause(); movieC.Pause()
mgr.EndBurst()

// Per-frame, mixed compositing:
exp.Screen.Clear()
mgr.DrawWithoutFlip()           // decode + render movies
fix.Present(exp.Screen, false, false)  // overlay
ts, _ := exp.Screen.FlipTS()
mgr.NotifyFlipped(ts)
```

Onset precision summary

Source	Where	Meaning
`LookAhead`	`OnAt` / `OnDone` callbacks	`sdl.TicksNS()` at fire time, pre-vsync
`VsyncEstimated`	`OnAtDisplay` / `Display*` callbacks (fallback timer)	Post-`Present` `FlipTS`, ~vsync-period

`HardwareVerified`	`OnAtDisplay` / `Display*` callbacks (Stage 5 timer)	OS-measured first-pixel-visible, sub-ms

Platform status

Platform	Backend	Precision
macOS	`CVDisplayLink` (CoreVideo via purego)	`HardwareVerified`
Linux	`DRM_IOCTL_WAIT_VBLANK` (`/dev/dri/cardN`, `video` group)	`HardwareVerified`
Windows	(not yet — fallback)	`VsyncEstimated`
Other	(fallback)	`VsyncEstimated`

See MediaMovies.md §8 for the Windows roadmap and §7 for the trigger-synchrony contract (the wire pulse arrives ~0.5–5 ms after `Onset.TimestampNS`; calibrate the constant offset with a photodiode for sub-ms wire-side alignment).

Package `design`

Import: `github.com/chrplr/goxpyriment/design`

Data Structures

```
// Trial — one experimental trial
trial := design.NewTrial()
trial.SetFactor("condition", "congruent")
trial.GetFactor("condition")   // → "congruent"
trial.Copy()                   // deep copy

// Block — a sequence of trials
block := design.NewBlock("Practice")
block.SetFactor("type", "practice")
block.AddTrial(trial, copies, randomPosition)
block.ShuffleTrials()

// Experiment design (separate from control.Experiment)
exp.Design  // *design.Experiment — contains Blocks, DataVariableNames, etc.
```

Randomization

```
design.RandInt(a, b int) int                          // random int in [a, b]
design.RandElement(list []T) T                        // random element
 ↪  (generic)
```

```
design.CoinFlip(headBias float64) bool              // weighted coin flip
design.RandNorm(a, b float64) float64               // truncated normal in
 ↪  [a, b]
design.ShuffleList(list []T)                        // in-place Fisher-Yates
 ↪  shuffle (generic)
design.MakeMultipliedShuffledList(list []T, n int) []T  // n shuffled
 ↪  copies concatenated
design.RandIntSequence(first, last int) []int       // shuffled range
 ↪  [first, last]
```

Latin Square (Between-Subjects Counterbalancing)

```
// Registration
exp.AddBWSFactor("handedness", []interface{}{"left", "right"})

// At runtime — returns the condition for the current subject
condition := exp.GetPermutedBWSFactorCondition("handedness")

// Low-level
square, err := design.LatinSquare(elements, design.PBalancedLatinSquare)
square, err := design.LatinSquareInts(n, design.PCycledLatinSquare)
// permutation types: design.PBalancedLatinSquare, PCycledLatinSquare,
 ↪  PRandom
```

Package `clock`

Import: `github.com/chrplr/goxpyriment/clock`

```
clock.Wait(ms int)                    // block for ms milliseconds
clock.GetTime() int64                 // ms since package first used
clock.GetTimeNS() int64              // nanoseconds since package first used

c := clock.NewClock()                 // clock relative to "now"
c.NowMillis() int64                   // ms elapsed
c.NowNanos() int64                    // nanoseconds elapsed
c.Now() time.Duration
c.Reset()                             // restart reference
c.SleepUntil(target time.Duration)    // sleep until target offset (returns
 ↪  immediately if past)
```

> **Note:** Prefer `exp.Wait(ms)` over `clock.Wait(ms)` in experiment code — `exp.Wait` pumps SDL events and detects ESC.
>
> **Clock domains:** `GetTimeNS()` and `NowNanos()` use the Go monotonic clock (`time.Since`). SDL event timestamps from `Screen.FlipTS`, `GetKeyEventTS`, `GetPressEventTS`, and `WaitAnyEventTS` use `sdl.TicksNS()`. The two clocks

have different origins and **must not be subtracted from each other** for reaction-time computation. Use the SDL-based functions exclusively for RT measurement.

Package `geometry`

Import: `github.com/chrplr/goxpyriment/geometry`

```
geometry.GetDistance(p1, p2 sdl.FPoint) float32
geometry.CartesianToPolar(x, y float32) (radius, angleDeg float32)
geometry.PolarToCartesian(radius, angleDeg float32) (x, y float32)
geometry.DegreeToRadian(deg float32) float64
```

Package `triggers`

Import: `github.com/chrplr/goxpyriment/triggers`

Provides hardware TTL signal output (EEG/MEG trigger codes) and TTL input (response pads wired over serial). Lines are **0-indexed (0–7)** throughout; bit N of a bitmask corresponds to line N.

Interfaces

```
// OutputTTLDevice — send trigger codes to recording equipment.
type OutputTTLDevice interface {
    Send(mask byte) error                    // set all 8 lines from bitmask
    SetHigh(line int) error                  // drive line HIGH (0-indexed)
    SetLow(line int) error                   // drive line LOW  (0-indexed)
    Pulse(line int, d time.Duration) error   // HIGH for d, then LOW (blocks)
    AllLow() error                           // all lines LOW
    Close() error                            // AllLow + release port
}

// InputTTLDevice — read TTL inputs from response hardware.
type InputTTLDevice interface {
    ReadAll() (byte, error)                                              //
 ↪  bitmask of all lines
    ReadLine(line int) (byte, error)                                    // 0
 ↪  or 1 (0-indexed)
    WaitForInput(ctx context.Context) (mask byte, rt time.Duration, err
 ↪  error)
    DrainInputs(ctx context.Context) error
    Close() error
}
```

DLPIO8 (DLP-IO8-G, USB-CDC serial)

Implements both `OutputTTLDevice` and `InputTTLDevice`.

```
// Auto-detect (recommended): returns NullOutputTTLDevice if not found, no
 ↪  error.
out, portName, err := triggers.AutoDetectDLPIO8()
defer out.Close()
out.Pulse(0, 10*time.Millisecond)   // 10 ms pulse on line 0

// Manual:
d, err := triggers.NewDLPIO8("/dev/ttyUSB0")
defer d.Close()
d.Send(0b00000101)                  // lines 0 and 2 HIGH
mask, err := d.ReadAll()            // bitmask of all 8 input lines
mask, rt, err := d.WaitForInput(ctx)
```

DLPIO20 (DLP-IO20, USB-CDC serial)

Implements both `OutputTTLDevice` and `InputTTLDevice`. 5 V logic (the T4 is 3.3 V).

> **Untested on hardware** — written from the datasheet rev 1.1. Bring it up with `tests/test_dlpio20` and a multimeter before relying on it.

The device has 17 usable digital channels but the TTL interfaces are 8-bit, so interface lines 0–7 address a *window* of 8 channels — `AN0–AN7` for output and `AN8–AN13`, `RB6`, `RB7` for input by default. Channels outside the windows stay reachable through the channel-level methods.

```
// Auto-detect (recommended): returns NullOutputTTLDevice if not found, no
 ↪  error.
out, portName, err := triggers.AutoDetectDLPIO20()
defer out.Close()

// Manual, with a remapped output window:
d, err := triggers.NewDLPIO20("/dev/ttyUSB0",
    triggers.WithIO20OutputChannels(
        triggers.IO20_AN0, triggers.IO20_AN1, triggers.IO20_AN2,
 ↪  triggers.IO20_AN3,
        triggers.IO20_AN4, triggers.IO20_AN5, triggers.IO20_AN6,
 ↪  triggers.IO20_AN7),
    triggers.WithIO20PollInterval(5*time.Millisecond),
)
defer d.Close()

d.Send(0b00000101)                      // lines 0 and 2 → AN0, AN2
d.Pulse(0, 5*time.Millisecond)
mask, err := d.ReadAll()                // bitmask of the 8 input-window
 ↪  channels
```

```
// Any of the 20 channels, in or out of the windows:
d.SetChannelHigh(triggers.IO20_RA4)
v, err := d.ReadChannel(triggers.IO20_AN12)
```

Channel constants are `IO20_AN0`–`IO20_AN13`, `IO20_RA4`, `IO20_P5`–`IO20_P7` (relay drivers — not TTL, not readable), `IO20_RB6`, `IO20_RB7`.

Timing: the device has no write-all command, so `Send` issues one 5-byte packet per line — the 8 lines change over ~3.5 ms rather than simultaneously. Use `SetHigh` on a single line for trigger onsets. On Linux, lower the FTDI latency timer (16 ms default) before timing-sensitive reads: `echo 1 | sudo tee /sys/bus/usb-serial/devices/ttyUSB0/latency_timer`.

MEGTTLBox (NeuroSpin Arduino Mega TTL box)

Implements both `OutputTTLDevice` and `InputTTLDevice`. Provides 8 TTL output lines (D30–D37) and 8 TTL input lines for a FORP response pad (D22–D29).

```
box, err := triggers.NewMEGTTLBox("/dev/ttyACM0",
    triggers.WithResetDelay(2*time.Second),   // wait for Arduino boot
 ↪  (default 2 s)
    triggers.WithPollInterval(5*time.Millisecond),
)
defer box.Close()

// Output
box.Pulse(0, 5*time.Millisecond)    // pulse line 0
box.PulseMask(0b00000011, 5*time.Millisecond)  // pulse lines 0 and 1
box.Send(0b00000001)                // set line 0 HIGH, all others LOW

// Input (FORP response pad)
_ = box.DrainInputs(ctx)            // clear latched presses from previous
 ↪  trial
mask, rt, err := box.WaitForInput(ctx)
buttons := triggers.DecodeMask(mask)  // []FORPButton
```

`FORPButton` constants (also serve as 0-indexed line numbers for bitmask operations):

```
triggers.FORPLeftBlue    // 0
triggers.FORPLeftYellow  // 1
triggers.FORPLeftGreen   // 2
triggers.FORPLeftRed     // 3
triggers.FORPRightBlue   // 4
triggers.FORPRightYellow // 5
triggers.FORPRightGreen  // 6
triggers.FORPRightRed    // 7
```

FT232HTrigger (Adafruit FT232H, Linux)

Implements both `OutputTTLDevice` and `InputTTLDevice`. Pure-Go driver via Linux usbfs — no libftdi or D2XX installation required.

Wiring: AD0–AD7 (D-bus) → 8 TTL output lines; AC0–AC7 (C-bus) → 8 TTL input lines.

```
box, err := triggers.NewFT232H(
    triggers.WithFT232HPollInterval(5*time.Millisecond), // optional;
 ↪  default 5 ms
)
if err != nil { log.Fatal(err) }
defer box.Close()

box.Send(0b00000001)              // AD0 HIGH (persistent)
box.Pulse(0, 5*time.Millisecond)  // AD0: HIGH for 5 ms, then LOW
box.AllLow()

_ = box.DrainInputs(ctx)
mask, rt, err := box.WaitForInput(ctx)

// Auto-detect (falls back to NullOutputTTLDevice if no device found):
out, err := triggers.AutoDetectFT232H()
defer out.Close()
```

Prerequisites (Linux): - The `ftdi_sio` kernel module must not hold the device: `sudo rmmod ftdi_sio` - User needs rw access to `/dev/bus/usb/BBB/DDD`. Recommended udev rule: `ACTION=="add", SUBSYSTEM=="usb", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6014", MODE="0666", GROUP="plugdev"`

LinuxGPIOTrigger (Raspberry Pi and other Linux SBCs)

Implements both `OutputTTLDevice` and `InputTTLDevice`. Uses the Linux GPIO character device v2 API (kernel $\geq$ 5.10) — no external libraries required.

Wiring: any 8 GPIO pins for output, any other 8 for input. Pin numbers are chip-relative offsets (= BCM numbers on Raspberry Pi).

```
box, err := triggers.NewLinuxGPIOTrigger(
    triggers.WithGPIOOutputPins([8]int{17, 27, 22, 5, 6, 13, 19, 26}),
    triggers.WithGPIOInputPins([8]int{12, 16, 20, 21, 4, 25, 24, 23}),
    triggers.WithGPIOChip("/dev/gpiochip0"),           // optional; this is
 ↪  the default
    triggers.WithGPIOPollInterval(5*time.Millisecond), // optional; default
 ↪  5 ms
)
if err != nil { log.Fatal(err) }
defer box.Close()

box.Send(0b00000001)              // pin 17 HIGH
```

```
box.Pulse(0, 5*time.Millisecond)  // pin 17: HIGH for 5 ms, then LOW

_ = box.DrainInputs(ctx)
mask, rt, err := box.WaitForInput(ctx)
```

Output-only and input-only configurations are both valid — omit `WithGPIOOutputPins` or `WithGPIOInputPins` as needed.

Prerequisites: - Linux kernel ≥ 5.10 - User in the `gpio` group: `sudo usermod -aG gpio $USER`

LabJackT4 (Modbus TCP)

Implements both `OutputTTLDevice` and `InputTTLDevice`. Pure-Go Modbus TCP driver — no LabJack SDK or system library required.

Wiring: the T4's digital lines are DIO4–DIO19. Output lines 0–7 → DIO4–DIO11 (FIO4–FIO7 screw terminals + EIO0–EIO3 on the DB15); input lines 0–7 → DIO12–DIO19 (EIO4–EIO7 + CIO0–CIO3). DIO0–DIO3 are the T4's dedicated analog inputs AIN0–AIN3 and cannot be used as digital lines. DIO4–DIO11 are *flexible I/O* that power up as analog inputs; `NewLabJackT4` switches them to digital mode (`DIO_ANALOG_ENABLE`) before setting directions.

```
box, err := triggers.NewLabJackT4("192.168.1.100",
    triggers.WithT4PollInterval(5*time.Millisecond), // optional; default 5
 ↪  ms
    triggers.WithT4Timeout(1*time.Second),           // optional; default 1
 ↪  s
    triggers.WithT4UnitID(1),                        // optional; default 1
    triggers.WithT4OutputBase(4),                     // optional; DIO of
 ↪  output line 0
    triggers.WithT4InputBase(12),                     // optional; DIO of
 ↪  input line 0
)
if err != nil { log.Fatal(err) }
defer box.Close()

box.Send(0b00000001)              // output line 0 (FIO4) HIGH
box.Pulse(0, 5*time.Millisecond)  // FIO4: HIGH for 5 ms, then LOW

_ = box.DrainInputs(ctx)
mask, rt, err := box.WaitForInput(ctx)
```

The host string may include the port number (`"192.168.1.100:502"`); default Modbus port 502 is appended automatically if omitted.

ParallelPort (Linux LPT)

Implements `OutputTTLDevice`.

```
ports := triggers.AvailableParallelPorts()      // scans /dev/parport0..3
pp := triggers.NewParallelPort("/dev/parport0")
if err := pp.Open(); err != nil { log.Fatal(err) }
defer pp.Close()
pp.Send(0b00000111)                             // lines 0,1,2 HIGH
pp.Pulse(0, 10*time.Millisecond)
status, _ := pp.ReadStatus()                  // Linux only: status register
```

Prerequisites: `sudo modprobe ppdev`; user in the `lp` group.

Null devices

`NullOutputTTLDevice` and `NullInputTTLDevice` are silent no-ops, safe to call without hardware. `AutoDetectDLPIO8` and `AutoDetectFT232H` return a `NullOutputTTLDevice` when no device is found.

Package `assets_embed`

Import: `github.com/chrplr/goxpyriment/assets_embed`

Provides embedded default assets as `[]byte` slices, ready for `FontFromMemory` or `PlaySoundFromMemory`:

```
assets_embed.InconsolataFont  []byte  // default monospace TTF font
assets_embed.BuzzerWav        []byte  // error/incorrect feedback sound
assets_embed.CorrectWav       []byte  // correct/reward feedback sound
```

Coordinate System

All stimulus positions are in **screen-center coordinates**:

- `(0, 0)` = screen center
- Positive X = right; positive Y = **up** (not down)
- `screen.CenterToSDL(x, y)` converts to SDL's top-left origin

```
        +Y (up)
         |
-X ------+------ +X
         |
        -Y (down)
```

Typical Experiment Structure

```go
package main

import (
    "log"
    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/design"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := control.NewExperimentFromFlags("My Experiment", control.Black,
    ↪  control.White, 32)
    defer exp.End()

    exp.AddDataVariableNames([]string{"block", "trial", "condition", "key",
    ↪  "rt_ms"})

    // Build design
    block := design.NewBlock("main")
    for _, cond := range []string{"left", "right"} {
        t := design.NewTrial()
        t.SetFactor("condition", cond)
        block.AddTrial(t, 10, false)
    }
    block.ShuffleTrials()
    exp.AddBlock(block, 1)

    err := exp.Run(func() error {
        exp.ShowInstructions("Press F for left, J for right.\n\nPress SPACE
    ↪  to start.")

        for bi, blk := range exp.Design.Blocks {
            for ti, trial := range blk.Trials {
                cond := trial.GetFactor("condition").(string)

                exp.Show(stimuli.NewFixCross(20, 2, control.White))
                exp.Wait(500)

                stim := stimuli.NewTextLine(cond, 0, 0, control.White)
                exp.Show(stim)

                key, rt, err := exp.Keyboard.WaitKeysRT(
                    []control.Keycode{control.K_F, control.K_J}, 3000,
                )
                if control.IsEndLoop(err) {
                    return control.EndLoop
                }
```

```
                exp.Data.Add(bi, ti, cond, key, rt)
                exp.Blank(500)
            }
        }
        return control.EndLoop
    })
    if err != nil && !control.IsEndLoop(err) {
        log.Fatalf("experiment error: %v", err)
    }
}
```